Sri Lanka Institute of Information Technology

IT2130 – Operating Systems and System Administration
Year 2, Semester 2- 2026

Practical Test Answer Submission

Student ID	IT24100064
Student Name	Abeysiri K.K.K.M.P
Campus/ Center name	Malabe
Specialization	Software engineering
Weekday/ Weekend	Weekend
Batch No	WE SE 1.1

Question 1

A small educational software company is designing a multi-process learning tool to demonstrate how tasks can be divided between processes in an operating system.

The system is designed such that:

- The parent process interacts with the user and collects input
- The child process performs a computational task independently

As part of this system, the parent will take a number from the user, and the child will generate a Fibonacci series up to that number.

This design demonstrates how process creation and task delegation work in operating systems.

Write a C program that performs the following:

1. The parent process should:
    - Prompt the user to enter a positive integer n
    - Create a child process using fork()
2. The child process should:
    - Generate and print the Fibonacci series up to n terms
    - Display its PID and PPID
3. The parent process should:
    - Wait for the child process to complete using wait()
    - Print a message indicating completion

```
  GNU nano 8.4
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>

int main(){
        int n;
        pid_t pid;

        printf("Enter a positive integer: ");
        scanf("%d", &n);

        if(n<=0){
                printf("Please enter a positive integer. \n");
                return 1;
        }

        pid = fork();

        if (pid<0){
                printf("Fork failed.\n");
                return 1;
        } else if(pid ==0){

                printf("Child Process: \n");
                printf("PID: %d\n", getpid());
                printf("PPID: %d\n", getppid());

                int a =0, b=1, next, i;

                printf("Fibonacci series up to %d terms: ", n);

                for (i=1; i<=n; i++){
                        printf("%d", a);
                        next = a+b;
                        a=b;
                        b=next;

                }

                printf("\n");
        }
        else{
                wait(NULL);
                printf("Parent process: Child has completed execution\n");
        }

        return 0;
}
```

```
kalpaabeysiri@m1-virtualbox:~/Labtest$ ./q1
Enter a positive integer: 5
Child Process:
PID: 6939
PPID: 6938
Fibonacci series up to 5 terms: 01123
Parent process: Child has completed execution
```

Question 2

A small educational software company is designing a multi-threaded learning tool to demonstrate how tasks can be executed concurrently within a single process using threads in an operating system.

In this system:

- The main thread (parent thread) interacts with the user and collects input
- A worker thread is created to perform a computational task independently

This design demonstrates how thread creation, parallel execution, and synchronization work in operating systems.

Write a C program using POSIX threads (pthreads) that performs the following:

1. Main Thread:
   - Prompts the user to enter a positive integer n
   - Creates a new thread using pthread_create()
2. Worker Thread:
   - Generates and prints all prime numbers up to n
   - Displays:
     - Thread ID (pthread_self())
     - Process ID (getpid())
3. Main Thread:
   - Waits for the worker thread to complete using pthread_join()
   - Prints a message indicating that the thread has finished execution

```
kalpaabeysiri@m1-virtualbox:~/Labtest$ ./q2
Enter a positive integer: 10

Worker Thread:
Thread ID: 277036270285216
Process ID: 7488
Prime numbers up to 10 are: 2 3 5 7
Main thread: Worker thread has finished execution.
```

```
  GNU nano 8.4
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <stdlib.h>

void *printPrimes(void *arg){

        int n = *(int *) arg;
        int i, j, isPrime;

        printf("\nWorker Thread: \n");
        printf("Thread ID: %lu\n", pthread_self());
        printf("Process ID: %d\n", getpid());
        printf("Prime numbers up to %d are: ",n);

        for (i=2; i<=n; i++){
                isPrime=1;

                for(j=2;j<i;j++){
                        if(i%j==0){
                                isPrime =0;
                                break;
                        }
                }
                if (isPrime ==1){
                        printf("%d ",i);
                }
        }
        printf("\n");
        return NULL;
}

int main(){

        int n;
        pthread_t thread;

        printf("Enter a positive integer: ");
        scanf("%d", &n);

        pthread_create(&thread, NULL, printPrimes, &n);

        pthread_join(thread, NULL);

        printf("Main thread: Worker thread has finished execution.\n");

        return 0;
}
```

Question 3

A software company is developing a multi-threaded parallel data analysis system designed to efficiently process large datasets by dividing the workload among multiple threads.

The system demonstrates how:

- A main thread manages input and coordination
- Multiple worker threads execute computations in parallel on different portions of the data
- Results are combined to produce the final output

This design helps illustrate parallelism, synchronization, and shared memory usage in operating systems.

Write a C program using POSIX threads (pthreads) that performs the following:

1. Main Thread:

- Prompts the user to enter:
    - The size of an integer array n
    - n integer elements
    - The number of threads t (where t ≤ n)
- Divides the array into t nearly equal parts
- Creates t worker threads using pthread_create()
- Passes each thread:
    - Its portion of the array
    - Indices defining its range

2. Worker Threads:

Each thread should:

- Compute the partial sum of its assigned subarray
- Count the number of elements in its segment
- Display:

- Thread ID using pthread_self()
- Process ID using getpid()
- Its assigned range (start index to end index)
- Its partial sum

3. Main Thread (Aggregation):

- Wait for all threads using pthread_join()
- Collect partial sums using shared variables (with proper handling)
- Compute:
  - The total sum
  - The average of all elements
- Display the final results

```
GNU nano 8.4                                                    q3.c

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

int arr[100], partial[100];

typedef struct{
        int start;
        int end;
        int id;
} Data;

void *findSum(void *arg){
        Data *d=(Data *)arg;
        int i,sum=0,count=0;

        for(i=d->start; i<=d->end;i++){
                sum+=arr[i];
                count++;
        }

        partial[d->id]=sum;

        printf("\nThread %d\n", d->id +1);
        printf("Thread ID: %lu\n",(unsigned long) pthread_self());
        printf("Process ID: %d\n", getpid());
        printf("Range: %d to %d\n", d->start, d->end);
        printf("Range: %d to %d\n", d->start, d->end);
        printf("Partial Sum: %d\n", sum);

        return NULL;
}
```

```c
int main(){

        int n,t,i;
        pthread_t th[100];
        Data d[100];
        int total =0;
        double avg;
        int start =0;

        printf("Enter array size: ");
        scanf("%d", &n);

        printf("Enter %d elements: \n",n);
        for(i=0; i<n; i++){

                scanf("%d",&arr[i]);

        }

        printf("Enter number of threads: ");
        scanf("%d", &t);

        if(t>n){
                printf("Threads must be less than or equal to array size.\n");
                return 1;
        }
```

```c
        int size = n/t;
        int extra = n % t;

        for(i=0;i<t;i++){
                d[i].start =start;
                d[i].end = start + size -1;

                if (extra>0){
                        d[i].end++;
                        extra--;

                }

                d[i].id=i;

                pthread_create(&th[i], NULL, findSum, &d[i]);

                start = d[i].end + 1;

        }

        for (i=0; i<t;i++){

                pthread_join(th[i],NULL);
                total += partial[i];

        }
```

```
        for (i=0; i<t;i++){

                pthread_join(th[i],NULL);
                total += partial[i];

        }

        avg = (double)total/n;

        printf("\nFinal Results\n");
        printf("Total sum = %d\n",total);
        printf("Average = %.2f\n",avg);

        return 0;

}
```

```
kalpaabeysiri@m1-virtualbox:~/Labtest$ ./q3
Enter array size: 8
Enter 8 elements:
1
2
3
4
5
6
7
8
Enter number of threads: 3

Thread 1
Thread ID: 248333980529056
Process ID: 7970
Range: 0 to 2
Range: 0 to 2
Partial Sum: 6

Thread 2
Thread ID: 248333972074912
Process ID: 7970
Range: 3 to 5
Range: 3 to 5
Partial Sum: 15

Thread 3
Thread ID: 248333963620768
Process ID: 7970
Range: 6 to 7
Range: 6 to 7
Partial Sum: 15

Final Results
Total sum = 36
Average = 4.50
```